

Test Plan Report

Papillon

Course: BIL-496

Date: 15 March 2026

1. Introduction	3
2. Project Scope	3
Project Title	3
Brief description of the system	3
Main Functionalities	3
3. Features to Be Tested	4
Feature 1: Security, Access, and Cryptography Management	4
Feature 2: Threat Detection and AI Analysis	4
Feature 3: Vulnerability and Attack Surface Management	4
Feature 4: Platform Operations and Reporting	5
4. Testing Methodology	5
Unit Testing	5
Integration Testing	6
System Testing	6
Performance Testing	6
User Testing	7
5. Test Environment	7
Hardware (PC, GPU, camera, sensors, etc.)	7
Software (OS, programming language, libraries, frameworks)	7
6. Test Cases	8
7. Risks and Limitations	11
Possible technical risks:	11
Dataset or hardware limitations:	12
8. Conclusion	12
9. Requirement Traceability Matrix	12

1. Introduction

This test plan outlines the testing strategy and quality assurance objectives for the project to ensure it meets all specified functional requirements (FR1-FR10). Its primary purpose is to verify the system's operational stability, the accuracy of its AI-driven threat detection modules, and the secure integration of external services such as the NIST NVD and Microsoft Graph APIs. By systematically evaluating core capabilities like attack surface reconnaissance, vulnerability tracking, and secure credential management, this document guarantees that Papillon delivers a reliable, professional-grade Security Operations Center (SOC) experience prior to final deployment.

2. Project Scope

Project Title

AI-Driven Cybersecurity Framework for Intelligent Threat Detection and Analysis

Brief description of the system

Papillon is an integrated cybersecurity platform designed to provide professional-grade security tools to everyday users. The system offers session-based authentication with MFA, cryptographic operations, a live CVE vulnerability feed, automated attack surface analysis, Microsoft Outlook integration for phishing detection, and AI-driven security evaluations.

Main Functionalities

- **Cryptographic & Secret Management:** Robust data encryption utilizing AES-256-GCM and RSA-2048 algorithms, integrated with HashiCorp Vault for secure credential and key storage.
- **Real-Time Vulnerability Tracking:** Automated fetching and parsing of the latest CVE data and CVSS scores via the NIST NVD API.
- **Attack Surface Reconnaissance:** Orchestration of various scanning tools (Nmap, DNS, SSL, WHOIS, and subdomain enumeration) to analyze target domains and IPs.
- **AI-Driven Threat Detection:** Machine learning modules for malware analysis (Random Forest), network intrusion classification (XGBoost), email phishing detection (ONNX), and password strength evaluation (RandomForest).
- **Third-Party Integration:** Microsoft Outlook integration via OAuth 2.0 to securely access and scan inboxes for potential phishing attempts.
- **Comprehensive Reporting:** Generation of downloadable security scans and AI detection reports in jsPDF format.

3. Features to Be Tested

The system features have been grouped into logical categories based on the functional requirements (FR1 - FR11) to ensure a structured testing approach.

Feature 1: Security, Access, and Cryptography Management

- **Security & Access (FR10):** Verification of Role-Based Access Control (RBAC) across Admin, Analyst, and ReadOnly roles. Testing the mandatory Two-Factor Authentication (2FA) for administrators and validating the generation of comprehensive audit logs for all system actions.
- **Cryptography Module (FR8):** Testing the built-in utility for secure AES-256 and RSA operations, including the reliable and secure storage of encryption keys within the Key Management System (KMS).

Feature 2: Threat Detection and AI Analysis

- **Malware Analysis (FR7):** Testing the file upload mechanism, secure execution within the sandbox environment, PE feature extraction, and the accuracy of the AI model in classifying malware families.
- **Phishing Detection (FR6):** Verifying OAuth connectivity to mailboxes, automated parsing of incoming emails, and ensuring the ML model scores phishing probabilities within the strict 30-second timeframe.
- **IDS Integration (FR4):** Testing the ingestion of Intrusion Detection System logs via webhooks, log normalization, and the AI-driven real-time detection and visualization of traffic anomalies.
- **Password Strength Detection (FR11):** Testing the password strength evaluation module by validating secure input processing, feature extraction including length, entropy, character variety and verifying the accuracy of the machine learning model in classifying passwords as weak, medium, or strong.

Feature 3: Vulnerability and Attack Surface Management

- **CVE Automation (FR2):** Ensuring the platform automatically fetches vulnerability data from NVD and ExploitDB at least every 12 hours, normalizes the data, and successfully matches it against user assets.
- **Attack Surface Analysis (FR3):** Validating the isolated and non-intrusive scanning of verified domains to discover subdomains, open ports, and verify SSL certificate validity.
- **Attack Simulation (FR5):** Testing the capability to trigger controlled attack scenarios within a sandboxed environment to evaluate system defenses, followed by the generation of automated reports.

Feature 4: Platform Operations and Reporting

- **Blacklist Management (FR1):** Testing CRUD (Create, Read, Update, Delete) operations for managing blocked IP addresses, ensuring strict IPv4 and IPv6 format validation, and verifying the UI visualization.
- **Reporting (FR9):** Validating the automated and manual generation of customizable PDF reports that accurately summarize security findings, alerts, and scan results for stakeholders.

4. Testing Methodology

To prove the architectural integrity and operational resilience of the project, isolated and unified testing strategies will be implemented for each of the defined functional requirements (FR1-FR10). In this context, the following testing methodologies have been orchestrated according to the structural necessities of the respective requirements:

Unit Testing

The verification of system components at an atomic level within memory spaces isolated from external dependencies.

- **Required for FR1 (Blacklist Management):** There is an imperative need to test the regex-based parsing engines, which audit the syntactic integrity and semantic validity of the ingested IPv4 and IPv6 addresses, with zero fault tolerance.
- **Required for FR8 (Cryptography Module):** Mathematical validation tests are essential to prove that the AES-256-GCM and RSA-2048 algorithms complete their encryption/decryption cycles without inducing any entropy loss or memory leaks.
- **Required for FR11 (Password Strength Detection):** Unit tests are required to validate the correct functioning of the password entropy calculation, length verification, and character diversity checks. The Random Forest model must consistently classify passwords as Weak, Medium, or Strong. Edge cases, such as very short or highly complex passwords, should be tested to ensure robustness.

Integration Testing

The auditing of asynchronous data pipelines and API handshake protocols between internal microservices and external service providers.

- **Required for FR2 (CVE Automation):** End-to-end data-path testing is mandatory to verify that heterogeneous data flowing from external sources like NIST NVD and ExploitDB is losslessly transferred to the normalization layer in the Django backend.
- **Required for FR6 (Phishing Detection):** It is required to validate the secure completion of the OAuth 2.0 authorization flow initiated via the Microsoft Graph API and the uninterrupted transmission of email data payloads to the ML (Machine Learning) microservices.

System Testing

The full-capacity simulation of the integrated architecture's entire lifecycle, from the user interface down to the database tier.

- **Required for FR3 (Attack Surface Analysis) and FR9 (Reporting):** A holistic testing approach is needed for the process of transforming the telemetry data-acquired by the Nmap, DNS, and SSL orchestration triggered in an isolated environment-into a customizable PDF report without any data corruption.
- **Required for FR5 (Attack Simulation):** It is vital to prove, at the system level, the efficacy of the isolation barriers that prevent controlled exploit scenarios executed within the sandboxed architecture from penetrating the platform's core system components.

Performance Testing

The evaluation of artificial intelligence inference engines and intensive data processing pipelines under bottlenecks and strict SLA constraints.

- **Required for FR6 (Phishing Detection) and FR4 (IDS Integration):** Stress and load tests are crucial to confirm that the ONNX and XGBoost runtimes do not exceed the critical 30-second inference latency threshold, even under high-volume network anomaly logs or email batches.
- **Required for FR7 (Malware Analysis):** It is necessary to measure that the Random Forest-based classification model does not experience statistical deviation in its Precision and Recall metrics under increasing concurrent file uploads.

User Testing

The validation of end-user interactions, usability heuristics, and access policies based on zero-trust architecture through real-world scenarios.

- **Required for FR10 (Security & Access):** To prove the impenetrability of the Role-Based Access Control (RBAC) matrix, penetration-focused usability tests simulating privilege escalation or MFA (Multi-Factor Authentication) bypass attempts by synthetic users with a "ReadOnly" profile are essential.
- **Required for FR1 (Blacklist):** It is necessary to verify the compliance of the CRUD operations' visualization responses in the UI and the stakeholder experience with established usability heuristics.

5. Test Environment

The testing environment is carefully configured to mirror the production setup. This ensures that all functional requirements, especially AI inferences and isolated security scans, are validated under realistic conditions.

Hardware (PC, GPU, camera, sensors, etc.)

- **Server Infrastructure:** Devices capable of simultaneously running backend services, databases, and continuous API requests will suffice.
- **AI & Inference Nodes:** Machines equipped with adequate processing power (and preferably dedicated GPUs) to ensure the Phishing (ONNX) and IDS (XGBoost) models meet the <30 seconds SLA.
- **Sandbox Environment:** A strictly isolated, air-gapped virtual machine or dedicated server solely for the safe execution of Malware Analysis (FR7) and Attack Simulations (FR5).
- **Client Devices:** Standard developer PCs and laptops (running Windows, macOS, or Linux) to validate the React-based User Interface across different screen sizes and browsers.

Software (OS, programming language, libraries, frameworks)

- **Operating Systems:** Ubuntu/Debian Linux for backend servers and sandbox environments; Windows/macOS/Linux for client UI testing.
- **Containerization & Orchestration:** Docker will be utilized to containerize the application components (Django backend, React frontend, Celery workers), and Kubernetes will be used to orchestrate these containers, ensuring a scalable and consistent test environment during intensive load testing.
- **Programming Languages:** Python 3.12.x (Backend and Machine Learning), JavaScript(Frontend).
- **Frameworks & Microservices:** Django (Core Backend), React & Vite (Frontend UI).

- **Databases & Message Brokers:** MySQL 8.0.x (Primary Data Storage), Redis/RabbitMQ (Task queuing for Celery workers).
- **Security & ML Libraries:** HashiCorp Vault (Key Management System), python-nmap, sslyze, ONNX Runtime, XGBoost, scikit-learn (Random Forest).
- **External Integrations:** Active internet connection configured for testing OAuth 2.0 (Microsoft Graph API) and fetching vulnerability feeds (NVD, ExploitDB).
- **Simulation Environment:** Virtual network environment using AWS or local VMs to simulate traffic between servers; MITRE CALDERA used for attack scenarios to test IDS, scanning, and anomaly detection modules.
- **Testing & UI Automation:** Selenium WebDriver will be used for automated end-to-end testing of the React frontend across multiple browsers (Chrome, Firefox, Edge). Selenium tests will be executed against the containerized frontend environment to validate UI workflows, OAuth 2.0 login flows (Microsoft Graph API integration), and dashboard interactions under various test scenarios.

6. Test Cases

The pass/fail criteria for the test cases dictate that a test is considered "Pass" only if the actual result perfectly matches the Expected Output, remains under the defined performance thresholds (p95), and yields no open P0 defects.

Test ID	Description	Input	Expected Output	Result
TC-01	End-to-end (E2E) latency analysis of IDS logs received via webhook using the Locust load testing tool.	Locust webhook payload generating 100 alerts/sec.	Time from webhook receipt to Dashboard display must be p95 ≤ 2.0 seconds .	<i>[To be executed]</i>
TC-02	Inference performance and accuracy analysis of the Phishing ML model.	Frozen test dataset retrieved via OAuth (Batch size 1).	Accuracy thresholds: Precision ≥ 0.90, Recall ≥ 0.80 . Inference latency: p95 ≤ 5.0 seconds (when GPU is available).	<i>[To be executed]</i>

TC-03	Malware PE feature extraction and Random Forest classification within the Sandbox environment.	Labeled PE (Portable Executable) dataset.	Parser Error Rate $\leq 1\%$. Classification model F1-Score ≥ 0.85.	<i>[To be executed]</i>
TC-04	Penetration testing and RBAC (Role-Based Access Control) privilege escalation audit.	Synthetic access requests to Admin endpoints by a user with a "ReadOnly" role.	0 successful bypasses. The system must strictly return a 403 Forbidden error.	<i>[To be executed]</i>
TC-05	Isolation and leakage control of the attack simulation.	Controlled exploit script triggered within the Sandbox environment.	No external network leakage; the report must be generated in strict isolation.	<i>[To be executed]</i>
TC-06	Reliability of scheduled CVE synchronization via NVD/ExploitDB.	Automated Scheduler trigger.	System logs must verify that synchronization runs completely and flawlessly every 12 hours .	<i>[To be executed]</i>
TC-07	Stress testing of dynamic PDF report generation with large datasets for stakeholders.	Database query containing approximately 1,000 rows of vulnerability and scan findings.	Customizable PDF report must be generated and downloaded in $\leq$ 15 seconds without memory leaks.	<i>[To be executed]</i>

TC-08	CRUD operations and input validation for Blacklist management.	Invalid format IPv4 and IPv6 strings (e.g., missing octets).	The system must reject the inputs and throw a UI error (verified by unit tests).	<i>[To be executed]</i>
TC-09	Cryptography module integration and HashiCorp Vault key management.	Plaintext credentials and AES-256 / RSA encryption cycle.	Encrypted and decrypted data must match exactly; Vault access policies must not be violated.	<i>[To be executed]</i>
TC-10	End-to-end Attack Surface Analysis (Nmap, DNS, SSL) usability and integrity testing.	Manual scan initiation on a verified target domain (INT-10 scenario).	Scans must be non-intrusive and isolated; users must complete the task with an ≥ 80% success rate without assistance.	<i>[To be executed]</i>
TC-11	Password feature extraction and Random Forest classification.	Password string samples with different complexity levels (e.g., "StrongP@ssw0rd!").	Model outputs strength_level: 2 (Strong).	<i>[To be executed]</i>
TC-12	End-to-end UI workflow validation using Selenium WebDriver across Chrome, Firefox, and Edge.	Automated Selenium scripts targeting the React frontend (login flow, dashboard, OAuth 2.0 integration).	All UI workflows complete without errors; OAuth 2.0 login redirects correctly; dashboard renders within p95 ≤ 3.0 seconds across all tested browsers.	<i>[To be executed]</i>

7. Risks and Limitations

During the testing lifecycle of the project, various risks that could impact operational continuity have been identified. To ensure the testing phases proceed without critical blockers, these risks and their corresponding mitigation strategies have been categorized below.

Possible technical risks:

- **External API Rate Limiting and Network Timeouts:**
 - Risk: The system strictly depends on third-party services such as the NIST NVD API, ExploitDB, and Microsoft Graph API (OAuth) for core functionalities like CVE Automation (FR2) and Phishing Detection (FR6). Aggressive automated testing pipelines can easily trigger "HTTP 429 Too Many Requests" errors, halting test executions and generating false-negative results.
 - Mitigation: The testing architecture will implement robust API mocking mechanisms. Instead of hitting live endpoints during every test cycle, the team will utilize recorded JSON data feeds, cached vulnerability responses, and mocked OAuth endpoints. Live API testing will be strictly reserved for low-frequency Integration Tests.
- **Asynchronous Task Bottlenecks:**
 - Risk: The platform utilizes an asynchronous worker architecture driven by Celery, RabbitMQ, and Redis to handle heavy scanning and AI inference tasks. Reproducing complex race conditions, Message Queue (MQ) overloads, or worker autoscaling failures within resource-constrained local test environments is technically highly improbable.
 - Mitigation: A prioritized testing approach will be adopted in local environments, focusing strictly on critical path tests (verifying core functional flows). Extreme stress tests and MQ overload scenarios will be deferred to subsequent sprints and executed exclusively within orchestrated staging environments (Docker/Kubernetes) that accurately mirror production capacity.
- **Sandbox Instability and Provisioning Delays:**
 - Risk: The execution of Malware Analysis (FR7) and Attack Simulations (FR5) relies entirely on a strictly isolated, air-gapped sandbox environment. Instabilities in hypervisors or delays in container provisioning processes can severely bottleneck security testing pipelines.
 - Mitigation: To avoid the overhead of full system provisioning for every test run, the QA team will prepare lightweight isolation tests and a "sandbox smoke test" checklist to validate containment logic efficiently.

Dataset or hardware limitations:

- **GPU Unavailability for AI Inference:**
 - Risk: The platform enforces strict performance SLAs, mandating that machine learning inference latencies (for the Phishing ONNX and IDS XGBoost models) remain strictly under $p95 \leq 5.0$ seconds. If dedicated GPU resources are unavailable in the continuous integration (CI) or local testing environments, meeting these baseline performance thresholds is hardware-impossible.
 - Mitigation: A secondary performance baseline will be established explicitly for CPU-only hardware setups. Fallback acceptance criteria will be defined, and any latency deviations resulting from the lack of GPU acceleration will be formally documented as "accepted hardware degradation" rather than logged as system defects.
- **Sensitive Data and PII Exposure:**
 - Risk: Given the nature of the cybersecurity domain, utilizing real-world datasets for testing IDS logs, phishing emails, or malware and password samples carries a massive hardware/data-centric risk of exposing Personally Identifiable Information (PII) or accidentally triggering live malicious payloads within the testing network.
 - Mitigation: Test datasets will be strictly limited to sanitized, anonymized, and synthetic data fixtures. Furthermore, all cryptographic keys and sensitive credentials required for the test environment will be securely managed via HashiCorp Vault (KMS) or restricted sample .env files, ensuring absolute zero hardcoded secrets in the source code.

8. Conclusion

This test plan provides a structured approach to testing the Papillon cybersecurity platform. By applying different testing methodologies ranging from unit tests of cryptographic functions to system tests of the React-Django integration, the platform's functionality, performance, and reliability are thoroughly evaluated. The results will indicate that the system meets the project requirements, successfully balancing trade-offs like security vs. performance, and is suitable for deployment.

9. Requirement Traceability Matrix

The Requirement Traceability Matrix (RTM) is a critical artifact that ensures all defined Functional Requirements (FR1 - FR11) are directly mapped to their respective Test Cases (TC). This matrix guarantees 100% test coverage, ensuring no system capability is left unverified before production deployment.

The functional requirements table is as follows:

Requirement ID	Requirement Description
FR1	The system must allow analysts to manage blocked IP addresses (CRUD operations) and visualize them, ensuring IPv4/IPv6 validation.
FR2	The platform must automatically fetch vulnerability data (from NVD/ExploitDB) at least every 12 hours, normalize it, and match it against the user's assets
FR3	Users need the ability to scan their verified domains for subdomains, open ports, and SSL validity. These scans must be isolated and non-intrusive.
FR4	The system must ingest logs from Intrusion Detection Systems via webhook, normalize them, and use AI to detect and visualize traffic anomalies in real-time.
FR5	Capability to trigger controlled attack scenarios in a sandboxed environment to test defenses, followed by automated reporting.
FR6	The system must connect to mailboxes (via OAuth), parse incoming emails, and score them for phishing probability using an ML model within 30 seconds.

FR7	Users must be able to upload suspicious files for analysis. The system executes these in a sandbox to extract PE features and classify the malware family.
FR8	A built-in utility for analysts to perform secure AES-256 and RSA operations, with secure key storage (KMS).
FR9	The ability to generate customizable PDF reports summarizing security findings, alerts, and scan results for stakeholders
FR10	Implementation of distinct roles (Admin, Analyst, ReadOnly) with mandatory 2FA for administrators and comprehensive audit logging for all actions.
FR11	Users will be able to test specific passwords using the password strength module to determine if a password is strong, and this will also be a useful model they can use during registration.

The requirement traceability matrix table is as follows:

Requirement ID	Requirement Description	Related Test Case	Coverage Status
FR1	Blacklist Management (CRUD & IPv4/IPv6 Validation)	TC-08	Covered
FR2	CVE Automation (NVD/ExploitDB Automated Sync)	TC-06	Covered
FR3	Attack Surface Analysis (Scans, DNS, SSL)	TC-10	Covered
FR4	IDS Integration (Webhook Ingestion & ML Anomalies)	TC-01	Covered
FR5	Attack Simulation (Sandboxed Exploit Execution)	TC-05	Covered
FR6	Phishing Detection (OAuth & ML Scoring within SLA)	TC-02	Covered

FR7	Malware Analysis (Sandbox Execution & Classification)	TC-03	Covered
FR8	Cryptography Module (AES/RSA & HashiCorp Vault)	TC-09	Covered
FR9	Reporting (Customizable PDF Generation)	TC-07	Covered
FR10	Security & Access (RBAC, 2FA, and Audit Logging)	TC-04, TC-12	Covered
FR11	Password Strength Detection	TC-11	Covered